



Republic of the Philippines
CENTRAL BICOL STATE UNIVERSITY OF AGRICULTURE
Impig, Sipocot, Camarines Sur 4408
Website: www.cbsua.edu.ph
Email Address: cbsua.sipocot@cbsua.edu.ph
Trunkline: (054) 881-6681



Unit 1

Web Programming

Basics

Lesson 5: Tailwind

Outline

Tailwind

- Tailwind CSS is a **utility-first framework** for building custom designs without custom CSS
- **Utility classes** are applied directly in HTML for quick layout creation
- Offers many utility classes for **responsive** and **custom** designs
- Supports **mobile-first** and responsive design development

Tailwind Setup

Getting Started with

Tailwind CSS



Using Tailwind via CDN

- Using Tailwind CSS via **CDN** requires no installation or setup
- Add the CDN link to the **<head>** of your HTML file
- Instantly start using Tailwind CSS utility classes in your project

```
<script src="https://cdn.tailwindcss.com/3.4.16"></script>
```

Installation Guide

[01] Install Tailwind CSS

Install `'tailwindcss'` and `'@tailwindcss/cli'` via npm.

Terminal

```
npm install tailwindcss @tailwindcss/cli
```

[02] Import Tailwind in your CSS

Add the `'@import "tailwindcss";'` import to your main CSS file.

src/input.css

```
@import "tailwindcss";
```

[03] Start the Tailwind CLI build process

Run the CLI tool to scan your source files for classes and build your CSS.

Terminal

```
tailwindcss/cli -i ./src/input.css -o ./src/output.css --watch
```

[04] Start using Tailwind in your HTML

Add your compiled CSS file to the `<head>` and start using Tailwind's utility classes to style your content.

src/index.html

```
<!doctype html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-s
  <link href="./output.css" rel="stylesheet">
</head>
<body>
  <h1 class="text-3xl font-bold underline">
    Hello world!
  </h1>
</body>
</html>
```

Container

Breakpoints in Tailwind CSS

Breakpoint	min-width
sm	640px
md	768px
lg	1024px
xl	1280px
2xl	1536px

Layout

Container

- By default, container does not center itself
- Add **mx-auto** to center it horizontally
- Add **px-*** for horizontal padding

Layout

mx-auto Class

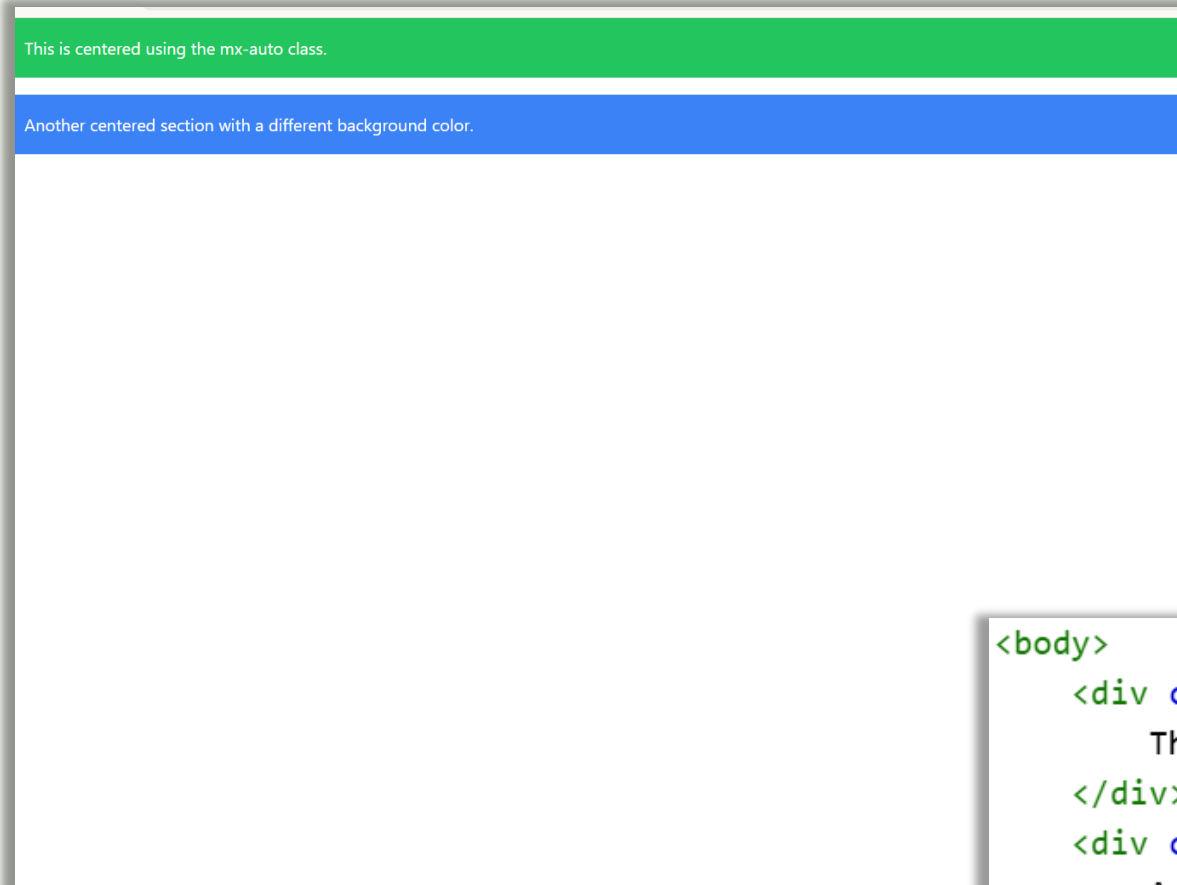
use to center a container by automatically applying horizontal margins.

Syntax:

```
<element class="mx-auto">...</element>
```

Layout

mx-auto Class



- **First div:** mx-auto centers the content horizontally
- **Second div :** mt-4 creates spacing between elements

```
<body>
  <div class="bg-green-500 text-white mx-auto p-4">
    This is centered using the mx-auto class.
  </div>
  <div class="bg-blue-500 text-white mx-auto p-4 mt-4">
    Another centered section with a different background color.
  </div>
</body>
```

Layout

px-{size} Class

use px-{size} to add horizontal padding equal to the specified size.

Syntax:

```
<element class="px-20">...</element>
```

Layout

px-{size} Class

```
<div class="bg-green-500 text-white mx-auto px-20">
```

This container has horizontal padding of 5rem (80px) on both sides.

```
</div>
```

This container has horizontal padding of 5rem (80px) on both sides.

- px-20 class applies **horizontal padding** of 5rem (80px) to the left and right sides of the container
- mx-auto class centers the container horizontally within its **parent element**

Box Sizing

box-border: Includes borders and padding in the element's **total width** and **height**.

box-content: Excludes borders and padding, calculating size only for the **content area**.

Box-Sizing

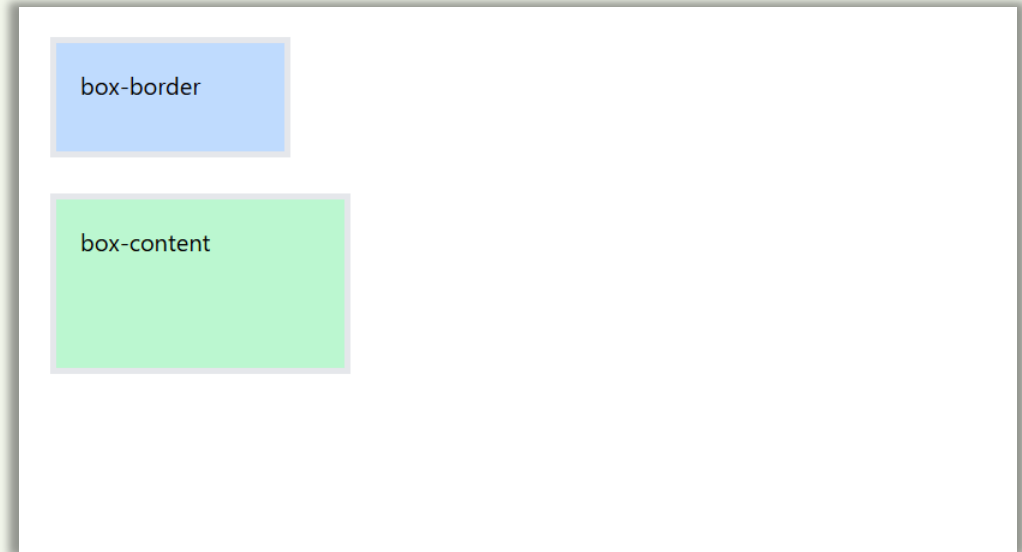
Box-Sizing Classes

```
<body>

  <!-- Box-border example -->
  <div class="box-border w-40 h-20 border-4 p-4 bg-blue-200 m-6">
    box-border
  </div>

  <!-- Box-content example -->
  <div class="box-content w-40 h-20 border-4 p-4 bg-green-200 m-6">
    box-content
  </div>

</body>
```



- The blue box (box-border) looks **smaller** overall (since padding + border are included in the size)
- The green box (box-content) looks **larger** (since padding + border are added on top of the width/height)
- **m-6** just adds some space around each box so they don't stick together

Object Fit

Object Fit

controls how an image or video fits within its container

Object Fit Classes

- object-contain
- object-cover
- object-fill
- object-none
- object-scale-down

Object Fit

object-contain Class

resizes content to fit within its container while preserving the aspect ratio

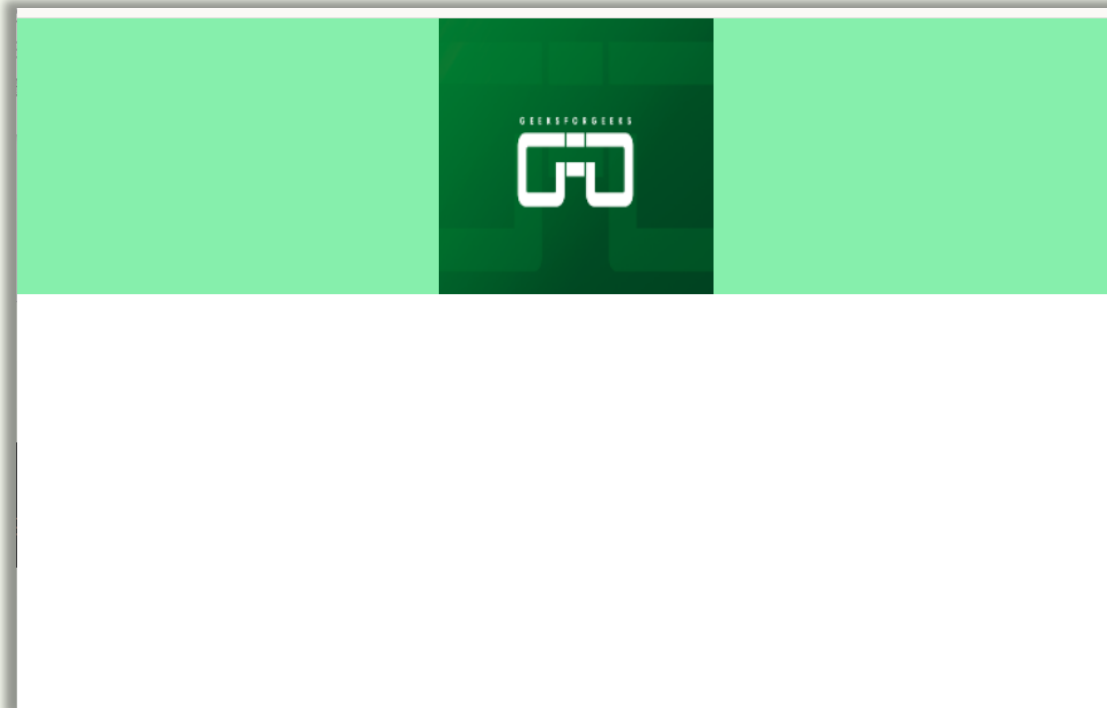
Syntax:

```
<element class="object-contain">...</element>
```

Object Fit

object-contain Class

```
<div class="bg-green-300 w-full h-full">  
    
</div>
```



object-cover Class

resizes content to cover its container while preserving aspect ratio, even if clipping occurs

Syntax:

```
<element class="object-cover">...</element>
```

Object Fit

object-cover Class

```
<div class="bg-green-300 w-full h-full">  
    
</div>
```



object-fill Class

stretches content to fully fill its container, ignoring aspect ratio

Syntax:

```
<element class="object-fill">...</element>
```

Object Fit

object-fill Class

```
<div class="bg-green-300 w-full h-full">  
    
</div>
```



Object Fit

object-none Class

displays content at its original size, ignoring both aspect ratio and container size.

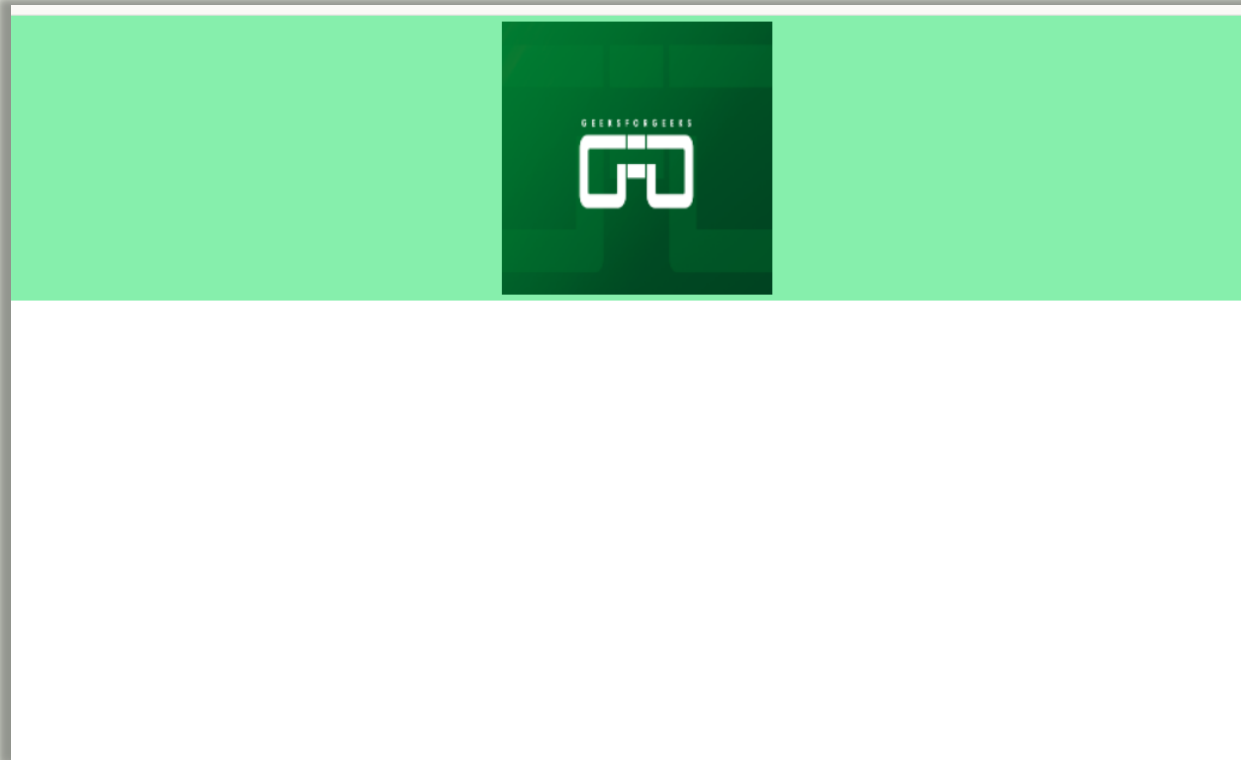
Syntax:

```
<element class="object-none">...</element>
```

Object Fit

object-none Class

```
<div class="bg-green-300 w-full h-full">  
    
</div>
```



Object Fit

object-scale-down Class

scales content down to fit the container, choosing the smaller size between none and contain

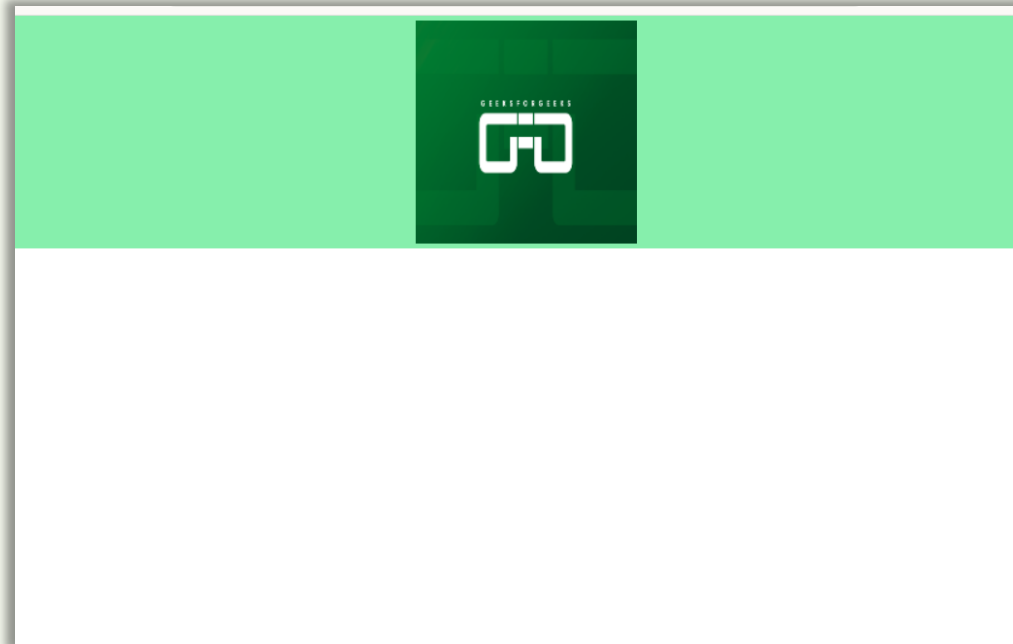
Syntax:

```
<element class="object-scale-down">...</element>
```

Object Fit

object-scale-down Class

```
<div class="bg-green-300 w-full h-full">  
    
</div>
```



z-index

z-index utility sets an element's stack order, equivalent to the CSS z-index property

Z-Index

z-index Classes

- z-0
- z-10
- z-20
- z-30
- z-40
- z-50
- z-auto

Syntax:

```
<element class="z-0">....</element>
```

Z-Index

z-0

used to give the z-index 0 to the element

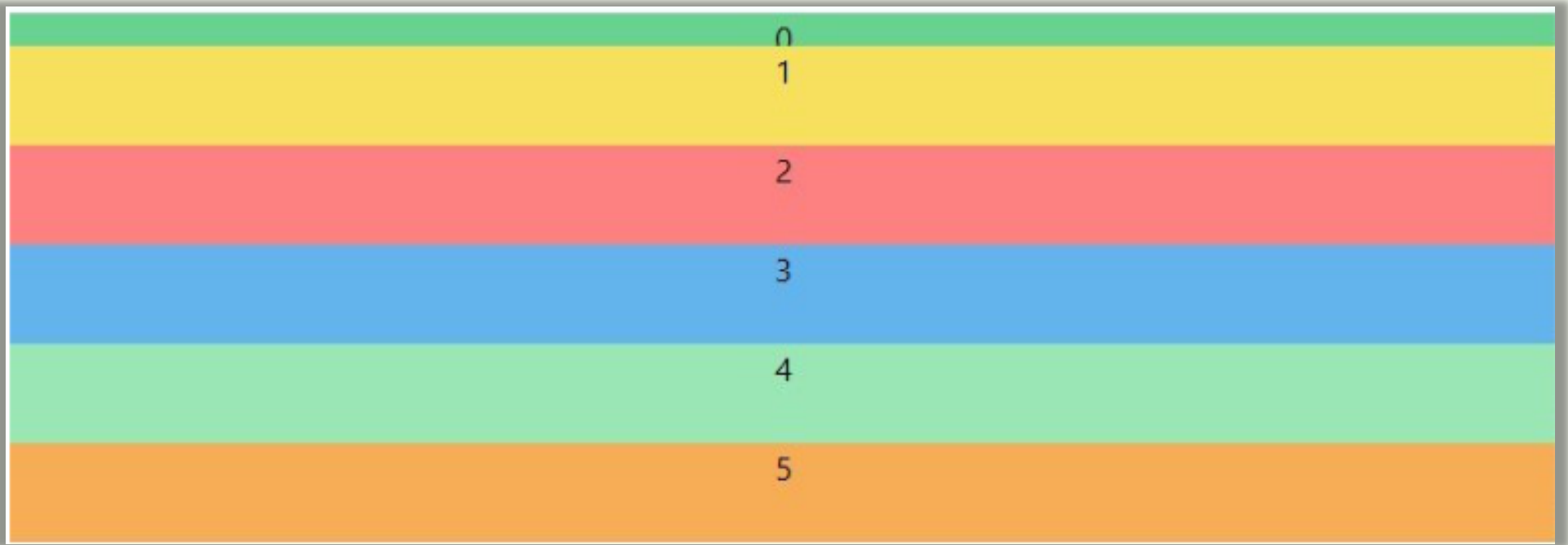
Syntax:

```
<element class="z-0">....</element>
```

```
<div class="flex-column">
  <div class="relative z-0 w-full
    h-12 bg-green-400"
    style="top: 32px;">0</div>
  <div class="relative z-10 w-full
    h-12 bg-yellow-400">1</div>
  <div class="relative z-20 w-full
    h-12 bg-red-400">2</div>
  <div class="relative z-30 w-full
    h-12 bg-blue-400">3</div>
  <div class="relative z-40 w-full
    h-12 bg-green-300">4</div>
  <div class="relative z-50 w-full
    h-12 bg-orange-400">5</div>
</div>
```

Z-Index

z-0



Z-Index

z-10

- It is used to give the *z-index* 10 to the element.
- The elements appear 10px upside along the z-axis.

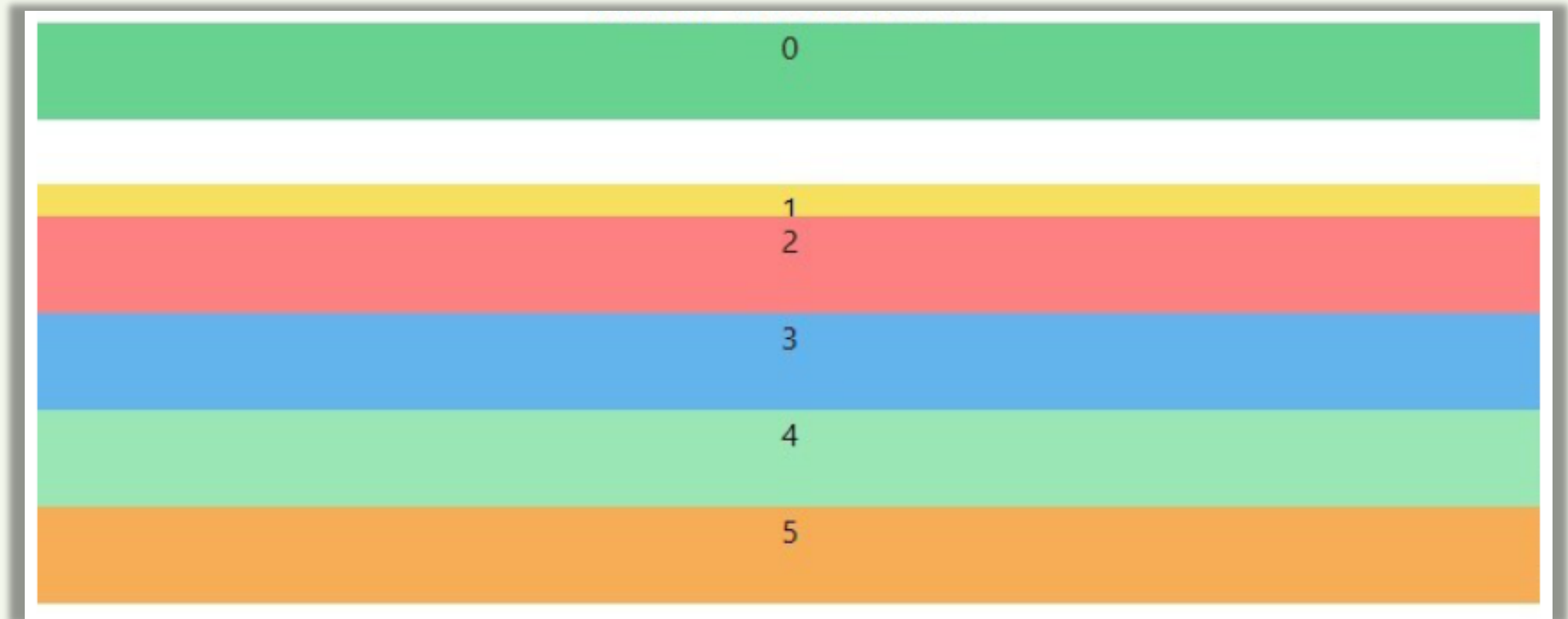
Syntax:

```
<element class="z-10">....</element>
```

```
<div class="flex-column">
  <div class="relative z-0 w-full h-12 bg-green-400">0</div>
  <div class="relative z-10 w-full
    h-12 bg-yellow-400" style="top: 32px;">1</div>
  <div class="relative z-20 w-full h-12 bg-red-400">2</div>
  <div class="relative z-30 w-full h-12 bg-blue-400">3</div>
  <div class="relative z-40 w-full h-12 bg-green-300">4</div>
  <div class="relative z-50 w-full h-12 bg-orange-400">5</div>
</div>
```

Z-Index

z-10



Z-Index

z-20

- It is used to give the *z-index* 20 to the element
- The elements appear 20px upside along the z-axis.

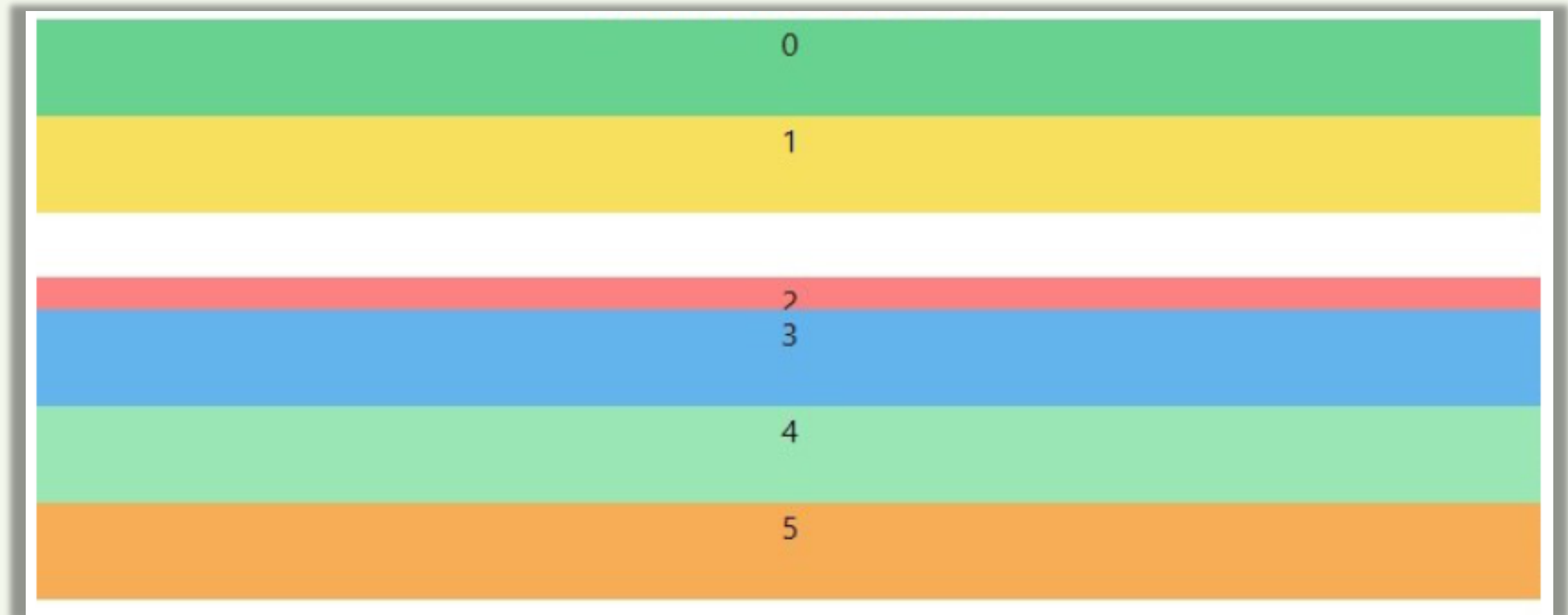
Syntax:

```
<element class="z-20">....</element>
```

```
<div class="flex-column">
  <div class="relative z-0 w-full h-12 bg-green-400">0</div>
  <div class="relative z-10 w-full h-12 bg-yellow-400">1</div>
  <div class="relative z-20 w-full h-12 bg-red-400"
        style="top: 32px;">2</div>
  <div class="relative z-30 w-full h-12 bg-blue-400">3</div>
  <div class="relative z-40 w-full h-12 bg-green-300">4</div>
  <div class="relative z-50 w-full h-12 bg-orange-400">5</div>
</div>
```

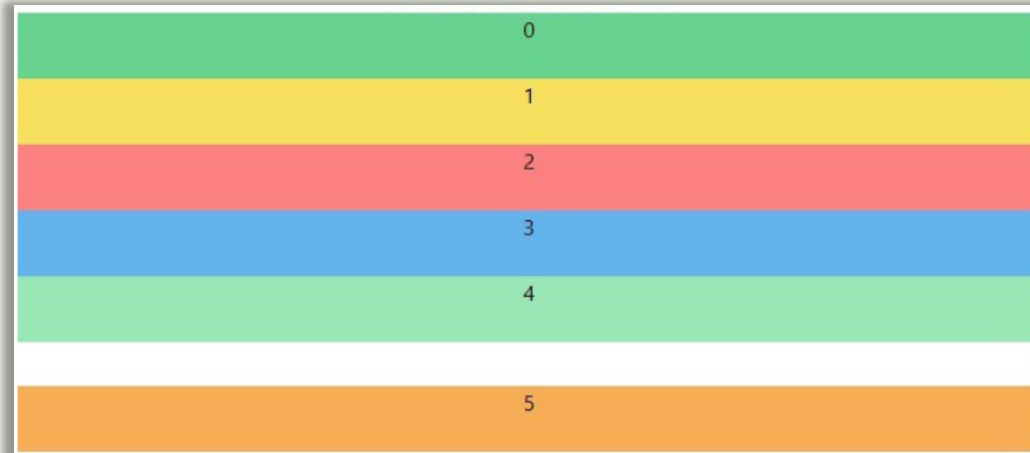
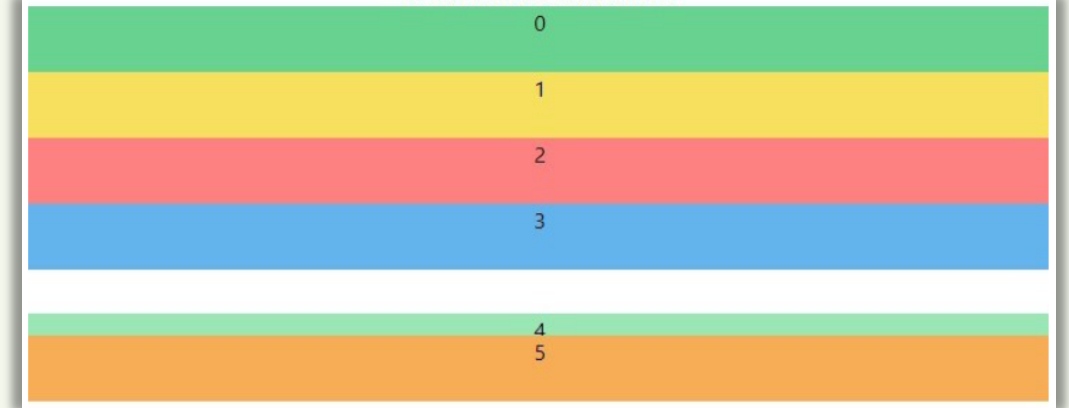
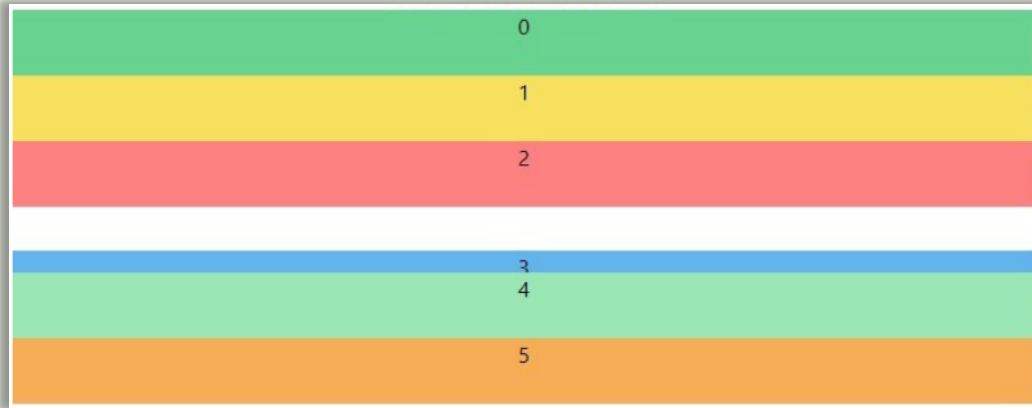
Z-Index

z-20



Z-Index

z-30, z-40, z-50



Z-Index

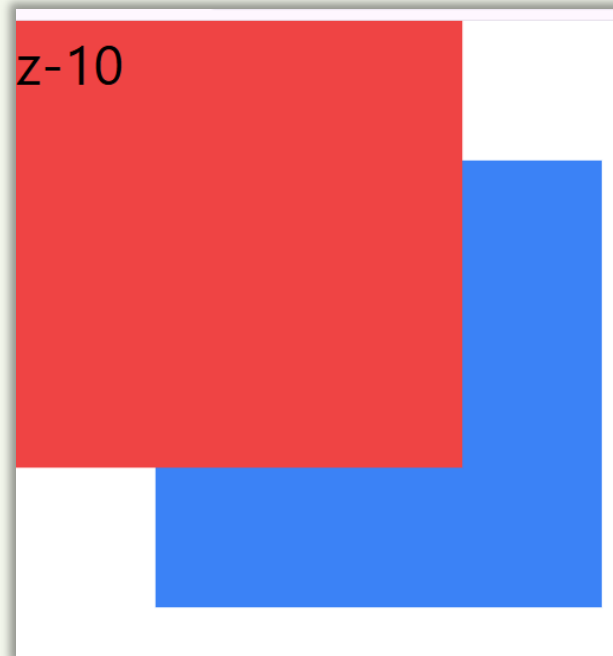
z-auto

z-auto sets the element's z-index to auto, letting it follow the natural stacking order without overriding it.

```
<div class="relative">  
  <div class="absolute top-0 left-0 bg-red-500 w-32 h-32 z-10">z-10</div>  
  <div class="absolute top-10 left-10 bg-blue-500 w-32 h-32 z-auto">z-auto</div>  
</div>
```

Z-Index

z-auto



- The red box uses z-10, so it appears on top of normal elements.
- The blue box uses z-auto, so it follows default stacking, typically appearing beneath the red box, assuming no other stacking context changes.

References

- <https://www.geeksforgeeks.org/css/tailwind-css/>